

Final Project Guide

Show the world you are a Junior Backend Developer

Congratulations — You Made It!

Over the last eight weeks you have gone from the command line all the way to building and running your own REST API with a live database. That is not a small thing. Today is your final day, and instead of an exam, you are going to prove what you know by building something real.

Why?

Employers and open-source projects hire junior backend developers based on what they can build — not just what they have memorised. Your final project is your first piece of portfolio evidence.

What You Have Learned (Your Toolkit)

Every skill below is something you can apply in your final project and point to in future job applications.

Week	Topic	Skills You Now Have
Week 1	Web Architecture & CLI	Client-server model, DNS, HTTP basics, Linux terminal navigation, Git & GitHub
Week 2	Linux & Python Intro	File permissions, directory structure, package managers (apt), bash scripts, Python setup
Week 3	Python Fundamentals	Variables, data types, conditionals, loops, functions, user input, basic testing
Week 4	OOP & Modules	Classes, objects, instance & class methods, modules, packages, error handling
Week 5	Databases & SQLAlchemy	Relational databases, SQL CRUD, SQLAlchemy ORM, environment variables (.env), MVC architecture
Week 6	APIs & FastAPI	REST API concepts, HTTP methods, endpoints, JSON responses, FastAPI, models, full CRUD API

What Your Project Must Show

Think of your final project as a mini-application a real business could use. It does not need to be complex — it needs to be correct and intentional. Your project must demonstrate all three layers of what you have learned:

Layer	What It Means For Your Project
Python Backend	Written in Python with clean, well-named variables and functions. Logic is separated into sensible functions or classes.
A Database	Data is stored in a real database (SQLite is fine). You use SQLAlchemy to connect and perform at least CREATE and READ operations. Data persists after a server restart.
A FastAPI	Accessible through a REST API with at least 3 endpoints. Accepts and returns proper JSON responses with correct status codes.
Good Structure	Organized folder structure with separate files for models, database setup, and routes. A .env file for config and a requirements.txt.
Git History	Code is on GitHub with at least 5 meaningful commit messages showing your work progressed over time.
A Demo Video	A short 2–3 minute screen recording walking through your project. Explains what it does, shows the /docs page and live API calls. Hosted on YouTube (unlisted is fine) or Loom.
A LinkedIn Post	A post announcing your project when it is complete. Template provided in this guide.
A Project Flyer	A single-page visual that explains your project to a non-developer. Design it using Canva or any AI design tool. Include the project name, what it does, your tech stack, and your GitHub/demo link.

Project Ideas to Get You Started

You are free to build anything you want. Here are ideas across difficulty levels — pick one that excites you, you will do better work on something you care about.

Project Idea	What It Involves	Level
Personal Task Manager API	Create, read, update, and delete tasks. Each task has a title, description, due date, and a status (pending / done).	Beginner
Student Grade Tracker	Record students and their scores per subject. Endpoints to add a student, add a grade, get a student’s average, and list all students.	Beginner
Recipe Book API	Store recipes with a name, ingredients, and steps. CRUD endpoints plus a search endpoint to find recipes by	Beginner

Project Idea	What It Involves	Level
	ingredient.	
Library Book Tracker	Manage books by title, author, genre, and availability. Allow marking a book as borrowed or returned.	Beginner
Contact Book API	Store contacts with name, phone, email, and group (friend, work, family). Filter contacts by group.	Beginner
Expense Tracker	Log daily expenses with amount, category, and date. Endpoint to get total spent per category. Proper SQLAlchemy model classes.	Intermediate
Simple Inventory System	Manage products for a small shop: name, price, quantity. Update stock when items are sold. Flag low-stock items.	Intermediate
Event Registration System	Create events and allow users to register. Track remaining spots. Return a message if an event is full.	Intermediate
Clinic Appointment Scheduler	Book appointments between patients and doctors. Check for time conflicts. List a doctor's schedule for the day.	Advanced

Tip

Pick a project that relates to something you know. If you love music, build a playlist tracker. If you are into food, build a recipe API. Your genuine interest will show in the quality of your code.

Where to Get Ideas for Bigger Projects

Whether for your university final year project, a portfolio piece, or future employment, the best project ideas come from real problems. Here is where to look:

Look at Problems Around You

- What frustrates you or people you know in daily life? A queuing system for a local clinic, a stock tracker for a family shop, a WhatsApp-style notification tool for a church group — these are all real backend problems.
- Think about your campus: student timetable management, hostel room booking, course registration — every university has processes that could be automated.
- Ghana-specific problems are valuable. A tool that manages market vendor inventory, tracks lorry station arrivals, or handles susu group savings is both relevant and impressive to local employers.

Open Source Project Boards

- [GitHub.com](#) — search topics like: python api beginner, fastapi good first issue, backend beginner-friendly
- [goodfirstissue.dev](#) — a curated list of beginner-friendly open source tasks across many languages
- [up-for-grabs.net](#) — another source of open source projects actively looking for new contributors

Hackathons & Challenges

- Devpost.com lists ongoing online hackathons — many are beginner-friendly and have prizes
- Follow Ghana Tech Lab, MEST Africa, and iSpace on social media for local hackathon announcements
- MLH (Major League Hacking) runs beginner-friendly hackathons throughout the year — you can join as a team

Final Year Project Repositories for Inspiration

- GitHub.com/topics/final-year-project — see what others have built and identify gaps you could improve
- Look at past projects from your own department. Ask your lecturers what problems the department or faculty still struggles with — those become excellent final year topics.
- IEEE Xplore and ResearchGate list academic project papers. You do not need to read the full paper — just the abstract and system design sections for ideas.

Domains With Strong Project Demand

- Health — appointment booking, patient record systems, medication reminders
- Education — result management, class attendance, student portal backends
- Finance — personal budgeting tools, susu / savings group trackers, invoice generators
- Agriculture — crop inventory, weather alert systems, farmer market price trackers
- Logistics — delivery tracking, driver management, warehouse stock systems

Minimum Requirements Checklist

Go through every item before you submit. If you cannot tick a box, that is the next thing to work on.

Project Setup

- Project lives in its own folder with a clear name (e.g., task-manager-api)
- A requirements.txt file listing fastapi, uvicorn, sqlalchemy, python-dotenv
- A .env file that stores the database connection string (never hardcoded in your code)
- A README.md that explains what the project does and how to run it

Python Code

- At least one Python class (model) representing your main data entity
- Functions are named clearly and each does one thing
- No unused variables or dead code left behind
- Error handling: the API returns a proper error if something goes wrong (e.g., item not found → 404)

Database

- SQLAlchemy used to define at least one table as a Python class
- The database is created automatically when the app starts
- At least CREATE and READ operations are working
- Data persists — stopping and restarting the server does not wipe your records

FastAPI Endpoints

- At least 3 endpoints (e.g., GET all, GET one by ID, POST to create)
- POST endpoint uses a Pydantic model to validate incoming data
- All endpoints return JSON with correct HTTP status codes
- The /docs page (Swagger UI) works and shows all your endpoints

Git & GitHub

- Repository is on GitHub
- At least 5 commits with meaningful messages (not just “update” or “fix”)
- README has a project title, tagline, and plain-language description of the problem it solves
- README has a complete “How to Run It Locally” section with every command
- README has an API Endpoints table showing method, path, and what each endpoint does
- README has at least one example request and example response so someone can test it without /docs
- If the API is hosted live, the live URL is in the README. If not, the local setup instructions are thorough enough that anyone can run it themselves.

Demo Video (Required)

- 2–3 minute screen recording showing your project running
- You speak over it or add captions explaining what each part does
- You show the /docs page and make at least one live API call
- You explain what problem your project solves in plain language
- Video is uploaded to YouTube (unlisted) or Loom and the link is in your README

LinkedIn Post (Required)

- You publish a post announcing your project when it is done
- Post includes what you built, what technology you used, a screenshot or the video link, and the GitHub link

Project Flyer (Required)

- A single-page visual design (A4 or square format) presenting your project
- Designed using Canva, Adobe Express, or any AI design tool
- Should be something a non-developer could read and understand in 30 seconds

How to Make a Great Demo Video

Most of your classmates do not have frontend experience, and that is completely fine. A backend API is invisible to the eye — but that does not mean you cannot show it off beautifully. Your demo video is how non-developers and future employers understand what you have actually built.

Tools

Use Loom (loom.com — free), OBS Studio (free), or even your phone propped up against your screen if needed. Loom is the easiest: install it, click record, and it auto-uploads.

What to Include in Your 2–3 Minute Video

Time	Section	What to Say / Show
0:00 – 0:30	Introduction	Tell us your name, what your project is called, and the problem it solves. Speak as if explaining to a friend who is not a developer.
0:30 – 1:00	The Problem	Show or describe the real-world situation your API addresses. E.g. “Right now, our school tracks grades in Excel spreadsheets. My API gives any app a proper way to store and retrieve that data.”
1:00 – 2:00	The API Demo	Open your browser and go to <code>http://127.0.0.1:8000/docs</code> . Show the endpoints. Click one, fill in the data, and execute it live. Show the response.
2:00 – 2:30	The Database	Open your GitHub repo and show the <code>models.py</code> or <code>database.py</code> file. Briefly explain what the table stores.
2:30 – 3:00	Wrap Up	Mention the GitHub link. Say what you learned or what you would add next if you had more time.

Remember

Non-developers do not understand code. They understand stories and problems. Always lead with the problem your project solves, not the technology it uses.

Showcasing Your Work Without a Frontend

You have built something real and technical — but most people outside of tech will not know what an API is. Here is how to make your work visible, understandable, and impressive to anyone who sees it.

1. Let FastAPI’s Built-in Docs Do the Work

FastAPI automatically generates a beautiful interactive documentation page at `/docs`. This is called Swagger UI and it is your best friend for showcasing your project:

- It shows all your endpoints with their descriptions

- Anyone can click “Try it out” and actually use your API in the browser — no code required
- It looks professional and is the industry standard for API documentation

Include a screenshot of your /docs page in your README, your LinkedIn post, and your demo video. This is the clearest way to show what your API does.

2. Write a README That Tells a Story

Your README is often the first thing someone reads. It should not just list installation steps — it should make someone understand why your project matters in 30 seconds. Structure it like this:

README Section	What to Write
Project Title & Tagline	Name and one sentence describing what it does. E.g. “Grade Tracker API — A backend system for managing student grades across subjects.”
The Problem	2–3 sentences on the real-world problem you are solving. No technical jargon.
What It Does	A short bulleted list of the main features. Use plain language.
Demo	Link to your Loom or YouTube video. Add a screenshot of /docs.
Tech Stack	Python, FastAPI, SQLAlchemy, SQLite, Uvicorn. One line each.
How to Run It Locally	Step-by-step setup instructions so anyone can clone and run your project. See the full template below — this is required if your API is not hosted live.
API Endpoints	A table listing every endpoint: method, path, what it does, and an example request body where needed.
How to Test It	Tell the reader exactly how to interact with the running API — via /docs in the browser, or with example curl/Postman commands. See examples below.

Rule	Never assume the reader knows anything. If there is a step that is obvious to you after 8 weeks, write it down anyway. The test is: can a friend who was not in this class follow these instructions without asking you a single question?
-------------	--

Show People How to Test Your API

Even if your API is not deployed, you can show people exactly what it does by including sample requests and responses directly in your README. There are two easy ways to do this:

Option A: Use the /docs Page (Easiest)

Tell the reader to go to `http://127.0.0.1:8000/docs` after starting the server. FastAPI’s Swagger UI lets anyone click “Try it out”, fill in a form, and see the live response — no extra tools needed. Include a screenshot of this page in your README so people know what to expect.

Option B: Include Example curl Commands

A curl command is a one-line instruction anyone can paste into their terminal to call your API. Include at least one example for each endpoint.

Alongside each command, paste the example JSON response your API returns. This way a recruiter, lecturer, or fellow developer can see exactly what your API produces without even running it:

Option C: Use Postman and Share the Collection

Postman is a free app for testing APIs. You can build a collection of all your requests inside Postman, then export it and include the file in your GitHub repo. Anyone who downloads your project can import your Postman collection and test every endpoint with one click.

- Download Postman at postman.com (free)
- Create a new collection and add one request per endpoint
- Fill in the URL, method, and body for each one
- Click Export → save the .json file into your project folder
- Add a line in your README: “Import postman_collection.json into Postman to test all endpoints”

Bottom line

If your project is not live, your README plus example requests IS your live demo. Write it as if you are handing it to someone who has never used an API before and wants to see it work.

3. Host Your API Live (Optional but Impressive)

Deploying your API means anyone in the world can access it from a link — without running anything on their laptop. These platforms offer free hosting for small projects:

- Railway (railway.app) — the easiest option for FastAPI with a database, free tier available
- Render (render.com) — good free tier, connects directly to your GitHub repo
- Fly.io — more technical but very powerful for Python apps

If you deploy, your /docs URL becomes a public link you can share anywhere. That is a portfolio piece any recruiter can click and explore.

4. Create a One-Page Project Summary

Prepare a one-page summary that non-developers can read. It should include:

- Your name and photo
- The project name and a one-sentence description
- A problem and solution paragraph
- A screenshot of the /docs page or a screen recording GIF
- The GitHub link and demo video link

Think of it as a poster, not a technical report. If someone’s parent or a business owner read it, would they understand what you built and why it matters? That is the test.

6. Take Good Screenshots

Screenshots are the fastest way to show your work. Take clean screenshots of:

- Your /docs page showing all endpoints (this is your “product screenshot”)
- A successful API response in the browser (shows it actually works)
- Your GitHub repo showing the commit history (shows your process)
- Your project folder structure (shows you know how to organise code)

Use a clean desktop background, zoom in so text is readable, and crop out anything irrelevant.

You are a backend developer.

Not someday. Now.

You know how the web works, how to write Python, how to build databases, and how to create APIs that other applications can consume. Build something with that knowledge and show it to the world.